

ClaimSense

Know what your insurance is actually worth: reconciling hospital bills against plan rules, one line item at a time.

Wireframes, AI / Tool Usage Disclosure

Tools used: Claude (Anthropic), Sonnet 5 model, for the wireframe layouts, the screenflow diagram, and a first pass at the design-rationale writeup. We also poked at a higher-fidelity version in Figma (through Claude's Figma integration) just to see the same layouts with real styling before settling on this low-fi version.

Usage: We fed Claude our ClaimSense_Project_Scope.docx (v0.2 draft) and asked it to sketch out wireframes for the screens we'd already agreed on as a team, then went back and forth a few times on layout and wording.

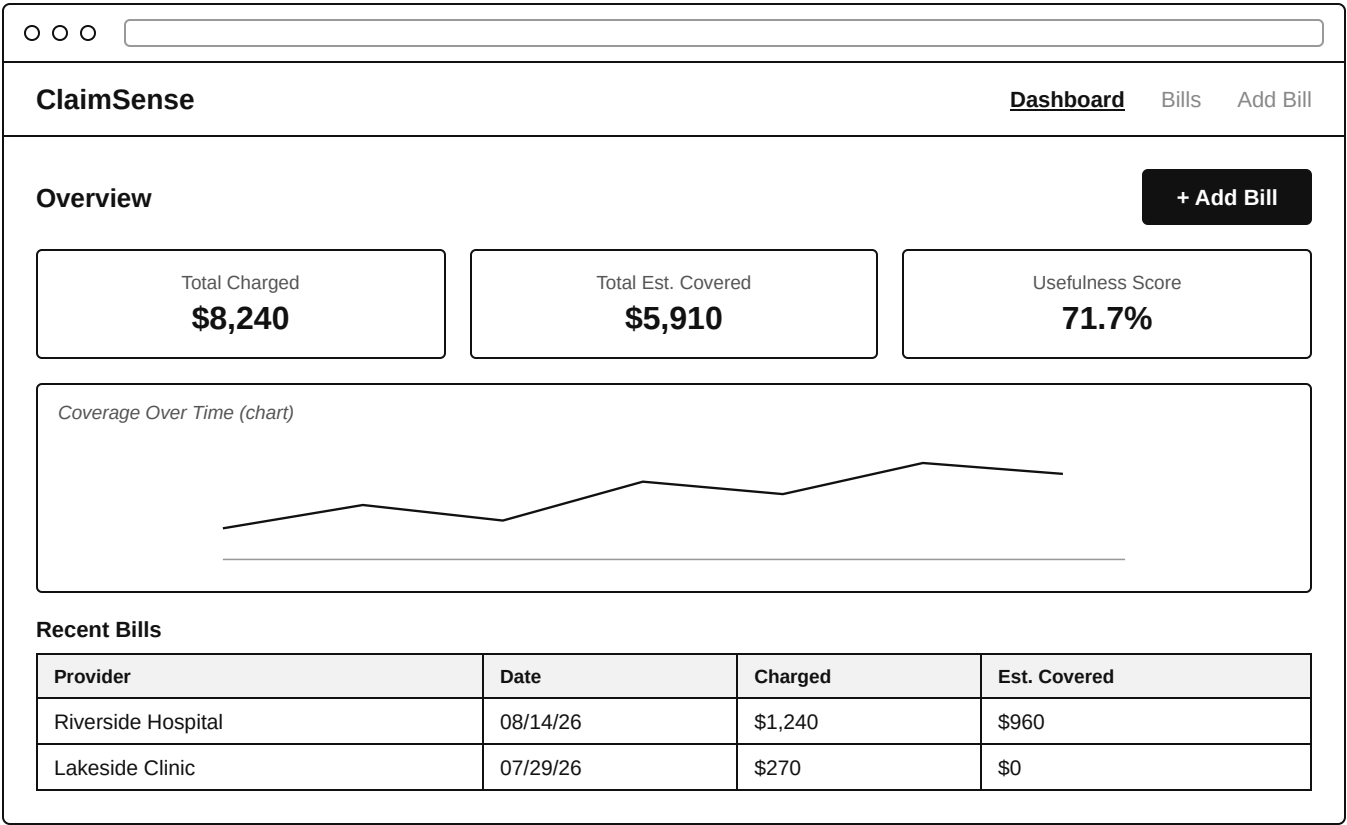
Human review: The whole team read through every page, checked it against what we actually scoped, and edited the copy before submitting.

Note on format

ClaimSense is a Django web app (list/detail pages, shared base template, JSON API) rather than a mobile app, so these wireframes use a desktop browser-window mockup instead of a phone frame.

Screen 1 of 4: Dashboard

ClaimSense, Low-Fidelity Wireframe (desktop web)



Key UI Elements

- Total Charged / Total Est. Covered / Usefulness Score stat cards
- Coverage-over-time chart (server-rendered Matplotlib PNG)
- Recent Bills mini-table
- + Add Bill primary action

Intended Interactions

- Stat cards and chart recompute from all of the user's logged bills on page load.
- Clicking a Recent Bills row opens that bill's Coverage Breakdown page.
- + Add Bill jumps straight to the Add Bill form.

PAGE PURPOSE

This is the first page a user lands on. Total charged vs. covered, the coverage trend, and the usefulness score are all visible without scrolling, so "is my plan actually good?" has a real answer in about two seconds.

Screen 2 of 4: Bills, Search & Filter

ClaimSense, Low-Fidelity Wireframe (desktop web)

ClaimSense

DashboardBillsAdd Bill

Bills

+ Add Bill

Provider

From Date

To Date

Apply

Totals by Provider

Riverside Hospital: \$1,450 charged / \$1,148 est. covered · Lakeside Clinic: \$270 / \$0

Totals by Month

Aug 2026: \$1,240 / \$960 · Jul 2026: \$270 / \$0 · Jun 2026: \$210 / \$168

Bill Ref	Date	Provider	Charged	Est. Covered	Patient Resp.
RH-1042	08/14/26	Riverside Hospital	\$1,240	\$960	\$280
LC-0087	07/29/26	Lakeside Clinic	\$270	\$0	\$270
RH-0951	06/05/26	Riverside Hospital	\$210	\$168	\$42

Key UI Elements

- Provider / date-range filter fields + Apply
- Grouped aggregate totals, by provider and by month
- Sortable bills table (bill ref, date, provider, charged, est. covered, patient responsibility)
- + Add Bill primary action

Intended Interactions

- Apply re-queries the table and both aggregate groupings via Django ORM filtering/aggregation.
- Clicking any table row opens that bill's Coverage Breakdown page.
- Column headers (not shown fully at this fidelity) toggle sort order.
- The same filtered bills and coverage outcomes are also available through the JSON Claims API, for anything outside the dashboard.

PAGE PURPOSE

This is where someone goes to find a specific bill, or check totals grouped by provider or by month, before opening any one bill's full breakdown. It's also the data the Claims API mirrors, just filterable through a browser instead of a request.

Screen 3 of 4: Add Bill

ClaimSense, Low-Fidelity Wireframe (desktop web)

ClaimSense

DashboardBillsAdd Bill

Log a New Bill

Patient

Reference Number

Provider (in-network for patient's plan)

Bill Date

Line Items

Line #	Description	Procedure Ref.	Charged Amount
1			
2			

+ Add Line Item

Save Bill

Key UI Elements

- Patient / Reference Number / Provider / Bill Date header fields
- Dynamic line-item table (line #, description, procedure reference, charged amount)
- + Add Line Item row control
- Save Bill primary action

Intended Interactions

- + Add Line Item appends another row with the next line number without leaving the page.
- The Provider field only lists providers on file for the patient's plan, since network status is tracked per plan.
- Estimated covered amount and patient responsibility per line item are filled in afterward in Django Admin; automatic calculation from plan rules is the next build step.
- On success, the user lands on that bill's Coverage Breakdown page.

PAGE PURPOSE

Where a user logs a new bill by hand: patient, a reference number, provider, date, and one numbered row per charge (description, procedure reference, charged amount). That's exactly what the Bill and BillLineItem models need to exist; the coverage math itself gets filled in as a follow-up step for now.

Screen 4 of 4: Bill Detail, Coverage Breakdown

ClaimSense, Low-Fidelity Wireframe (desktop web)

ClaimSense

DashboardBillsAdd Bill

< Back to Bills

RH-1042 · Riverside Hospital · 08/14/2026

#	Description	Procedure Ref.	Charged	Est. Covered	Patient Resp.
1	Emergency room visit	ER-99284	\$620	\$368	\$252
2	Lab work, CBC panel	LAB-85025	\$140	\$112	\$28
3	Attending physician fee		\$480	\$480	\$0

Plan Snapshot, Campus Student Health Plan (PPO)

Deductible: \$1,500 | Coinsurance: 20% | Out-of-Pocket Max: \$5,000

Provider network status under this plan: In Network

Total: \$1,240 charged / \$960 est. covered / \$280 patient responsibility

Key UI Elements

- Per-line-item charged/est. covered/patient-responsibility table, with procedure reference
- Plan snapshot: plan type, deductible, coinsurance percentage, out-of-pocket max
- Provider's network status under this specific plan
- Bill-level totals

Intended Interactions

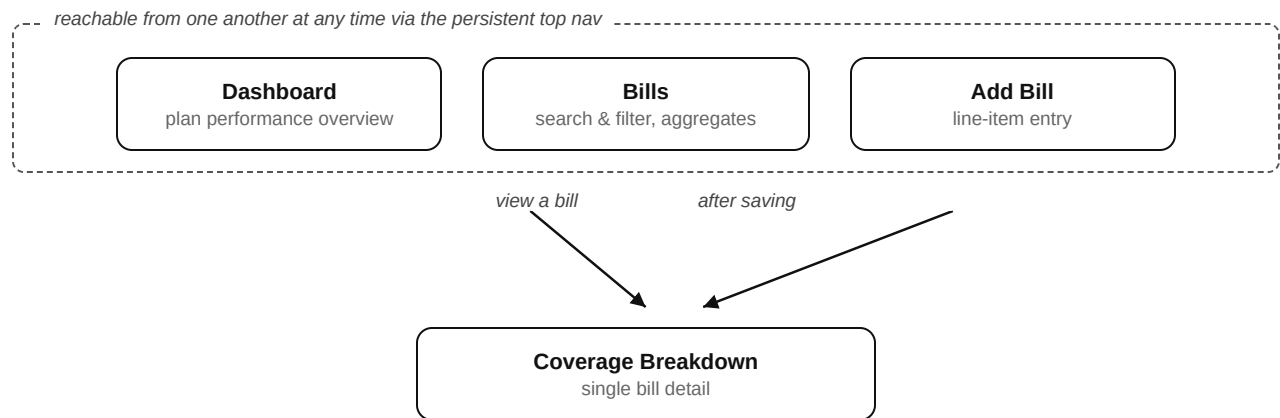
- Reached from a Bills-table row click, or immediately after Add Bill is submitted.
- Plan snapshot reflects the InsurancePlan record linked to this bill at the time it was logged.
- Back to Bills returns to the filtered list state the user came from.

PAGE PURPOSE

The receipts page, basically. It shows how one bill's charges line up against the plan on file, deductible, coinsurance, out-of-pocket max, and the provider's network status under that plan, so the covered/patient-responsibility split is never a guess.

Screenflow

ClaimSense, Navigation Overview



Every screen hangs off one persistent top nav with three destinations: Dashboard, Bills, and Add Bill. That's a direct match to the three independent tasks we scoped for this course. Bills and Add Bill both funnel into the same Coverage Breakdown page, one by clicking into an existing bill, the other right after logging a new one, so there's only one place in the whole app where someone actually reviews the reconciliation math. The plan itself gets entered through Django Admin, per the tech stack section of our scope doc, so we didn't need a separate Plan Setup screen in the user-facing flow at all. The Bills screen's filtering and aggregation also back a fifth, non-visual "destination": the JSON Claims API, which returns the same filtered bills and coverage outcomes for use outside the dashboard.

Design Documentation

Design Choices, Principles, and Justification

Hick's Law

We kept the top nav to 3 destinations (Dashboard, Bills, Add Bill), which happens to be exactly the number of user-facing screens we scoped. Fewer top-level choices means less time spent deciding where to click, which matters for a task that's already kind of tedious (reconciling a bill).

Fitts's Law

The actions people actually need to hit (+ Add Bill, Apply, Save Bill) are drawn as large, filled buttons instead of plain text links. Those are the ones worth making easy to click.

Gestalt Principles (Proximity & Similarity)

We reused one table style for line items, bill lists, and search results, so related data is grouped the same way no matter where you're looking at it. The three Dashboard stat cards use identical styling too, so they read as one comparable set instead of three unrelated boxes.

Cognitive Load Theory / Progressive Disclosure

The Bills list only shows the bill reference, date, provider, and totals. The full per-line-item math (procedure reference, plan deductible, coinsurance, out-of-pocket max, provider network status) is saved for the Coverage Breakdown page, for the one bill someone actually wants to dig into.

Jakob Nielsen's Heuristics

Visibility of system status: Coverage Breakdown shows the plan's deductible, coinsurance percentage, and out-of-pocket max right next to the per-line-item math, not just the final total. Match between system and the real world: the line-item table reads left to right the same way an actual itemized bill does, procedure reference and all. User control and freedom: filters can be reapplied or cleared any time without losing the underlying data.

Norman's Design Principles (Affordance & Feedback)

Saving a bill drops you straight onto its Coverage Breakdown, which is visible proof the calculation actually ran. Buttons are drawn with clear borders and fills so they read as clickable even at this fidelity.

Simplicity, Consistency, Accessibility

One page layout (window frame plus top nav) and one table style repeat across Dashboard, Bills, Add Bill, and Coverage Breakdown, so once you've seen the first screen, the rest of the app basically explains itself.

Design Documentation (cont.)

Layout, Flow, and Interaction Justification

Why the three core screens are peers, not a hierarchy

Dashboard, Bills, and Add Bill all sit on one nav because the three things they do (summarize, search, enter data) are tasks someone reaches for in whatever order makes sense to them. Logging a new bill has nothing to do with browsing old ones, so neither one should have to gate the other.

Why Bills and Add Bill both lead to the same Coverage Breakdown

Whether you're looking at a bill you logged weeks ago or one you just submitted, you're asking the same question: what did this actually cost me, line by line. Routing both paths into one shared page means we're not building (and maintaining) two versions of the same reconciliation view.

Why filters and aggregates sit above the table, not inside it

We treated provider/date filtering and the totals that come out of it as one query you set up once and read as a summary line before scanning the actual rows. That matches how Django's ORM aggregation works anyway: filter, then summarize. It's also why the Claims API takes the same provider/date filters as the page: the JSON endpoint and the dashboard are two views onto one query, not two separate features.

Why the plan itself doesn't get its own screen

Per our scope doc, plan entry and validation happen through Django Admin, not a user-facing form, and we're only supporting one active plan per user for this course. A separate in-app Plan Settings screen would just duplicate what Admin already does, and it would compete with Dashboard for "first thing you see after logging in," which the scope already settled for us.